CS 549 · Assignment 3: MVP Implementation and Demo

Last One Awake, a 3D stealth game about a cat wrecking an apartment at 3 a.m.
Group: Rushi Patel (leader), Dharm Patel, Antara Thapliyal, Christina Ross · Mentor: Danrui Li
Pillars: Physics & Dynamics, Collision Detection, NPC AI · This report describes the repository as of Oct 13, 2026.

> **Playable build:** **live URL to add** · To run it locally: clone the repo, run `python3 -m http.server 8550` in its root and open `localhost:8550/harness/play.html` (WASD to move, Space to jump, J to swat).
> **Video:** **YouTube link to add**
> **Source:** https://github.com/Antarathapliyal31/ai_visual_project · The README has the run instructions; `npm install && npm test` runs the 211 headless tests.

1. Plan vs. Reality

The slice we defined in Assignment 2 was the physics-to-AI path in one corner of the room: five stacked plates, two mugs and a book on a desk, the cat, and a sleeping owner who wakes in stages from the noise of real impacts. Its hardest part was a stack that rests stably while fast swatted objects never pass through thin geometry. **That hard part is built and measured. The path from impacts to the owner works only in an earlier single-file version and is not yet connected to the measured physics.**

The code is in two pieces that have not been merged:

- **Modular build** (`src/`, `harness/`; branches `main` and `Rushi`): our collision layer (Dharm) and our dynamics, contact solver and cat (Rushi), with 211 headless tests and three browser pages. It has no owner and no wake meter.
- **Single-file MVP** (`last-one-awake-mvp.html`; branch `antara`; Antara): cat, props, noise model, wake meter, and an owner who stirs, sits up, stands, walks to the cat and catches it. It uses stock cannon-es and its own 12 × 12 m room, and none of the modules above. The modular build has the 7 × 6 m room from our concept art.

What made it in

From the Assignment 2 MVP definition	Status	Evidence
Desk, five plates, two mugs, a book, the cat and the bed, seen through the isometric camera	**In**	Modular build, orthographic isometric view. Physics, ray queries and renderer all read the room from one list of boxes.
The cat walks, jumps onto the desk and swats	**In**	Jump apex 1.10 m (the desk is 0.94 m). Swat exit speed capped at 12 m/s. 65 tests.
Objects teeter, tip, fall and land on each other	**In**	78 rigid props, seven materials. A sliding box stops within 2% of the distance Coulomb friction predicts.
Impacts make noise, the wake meter fills, the owner stirs, sits up and stands	**Single file only**	Works in the single-file MVP. The modular build has no noise model and no owner.
Stacking proof. Five plates, sleeping off, 60 s: penetration under 2 mm, drift under 5 mm	**Pass**	0.07 mm and 0.02 mm, measured from the first step with no settling time. Same numbers in Node and in Chrome.
Tunneling proof. 100 swats at 12 m/s into a 2 cm shelf, none pass through	**Pass**	0 of 100 pass through with our swept-sphere guard, 22 of 100 without it. With the guard the first failure is above 100 m/s.
Coupling proof. The same swat wakes the owner further from beside the bed than from the far wall	**Not yet**	The single-file MVP's own test reads a wake level of 0.00 for both swats at its committed tuning, so it fails. The modular build has nothing to test yet.
Performance proof. 60 FPS with 60 awake bodies	**Partly**	60 FPS with 79 and with 151 awake bodies in the plain render. 50 to 53 FPS with our painted post-processing on, which triples the cost of drawing a frame. Apple M4, headless Chrome 155, 1658 × 856 pixels. Not yet run in Firefox or with the other proofs running.
Debug overlay, built first	**Changed**	There is no shared overlay. Each module exports its numbers and its own demo page draws them.

Built ahead of the slice (Assignment 2 had scheduled these after it): fracture of plates and mugs into pre-cut pieces that keep the parent's momentum, hiding volumes with line-of-sight queries, the whole furnished room, and lighting.

Limits we measured

- **Stack height.** 13 plates hold for 60 s. 14 walk off the pile (67 mm of drift).

- **Swat speed.** Without the guard a mug gets through the shelf at 12 m/s. With it, not until above 100 m/s.
- **Awake bodies, sleeping disabled.** One 120 Hz physics step takes 1.2 ms with 79 bodies, 3.3 ms with 151 and 5.6 ms with 251, and the plain render still holds 60 FPS. At 401 bodies a step takes 9.4 ms, two steps no longer fit a 16.7 ms frame, and the rate falls to 12 FPS. With sleeping on, the same render holds 60 FPS with 851 bodies at rest.

What was cut or changed

- **We wrote our own contact solver, which Assignment 2 listed under "won't have".** With cannon-es's stock solver the five plates drift 130 mm in our test, and 84 mm in the single-file MVP's own stacking test. Its source shows two causes: it starts every step from zero impulse, so a stack of three or more plates never converges, and it applies its friction limit per step, which makes friction 120 times too strong at 120 Hz. Our 319-line replacement adds warm starting, a true Coulomb friction bound, a restitution threshold and contact reduction to six points per pair. cannon-es still builds the contacts and integrates. We did not need the fallback to books.
- **All assets are procedural.** Assignment 2 planned glTF models, Blender lightmaps, a Mixamo owner and a pre-rigged cat. Instead every mesh is built in code from its collider's dimensions and every texture is drawn on a canvas at load time. The repository contains no model or texture files.
- **Noise is read from impact speed, not solver impulse.** The single-file MVP uses each contact's approach speed. The harvester that reads the solver's impulses, which is the coupling Assignment 2 described, is not written.
- **The owner is a stand-in.** In the single-file MVP his stage is a threshold on the wake level and, once standing, he walks straight to the cat. The state machine, vision cone and behaviour tree are not in the repository yet.
- **No build or deploy yet.** No Vite project, CI or live URL. The pages are plain ES modules served from a static server.

2. AI Utility

- **AI image tools for textures, and 3D generators (Meshy, Tripo, Rodin) for prop meshes: not used.** No AI-generated asset is in this build. Assignment 2 scheduled them after the slice, and so far code-built meshes have taken their place: a mesh made from its collider cannot disagree with it, and a broken plate can be drawn as pieces of the same profile.
- **Coding assistant (Claude Code): used heavily.** The collision layer and the dynamics, solver and cat code were written with it. All 14 commits of the collision layer carry its co-author line.
- **Runtime AI: none,** as specified.

Where it saved time. The plan gave the plate stack two weeks of parameter tuning. The assistant read the cannon-es solver source, found the two causes above within a day, and wrote the replacement with tests that compare it step for step against the stock solver. Most of the collision and dynamics tasks that the plan spread over Oct 12 to 28 were working by Oct 12.

Where it cost time. It does not know when it is wrong, so nothing counted until a test measured it. Two of its solver ideas did not help (one made the stack much worse) and were removed after measuring. Several of its own tests were wrong on the first run. A failing assertion on a physics body made the test runner print the whole world and run out of memory, which hid the real failure. It cannot observe frame rate in the browser tab it drives, so the painted look went in unmeasured and turned out to run 7 to 10 FPS under our target. It cannot judge whether a picture looks right: the look needed four rounds of correction from a person. Its additions to the shared room geometry still wait for that module owner's review, which our rule for AI-written collision code requires.

3. Roadmap to Final

- **One build (planned for Oct 22).** A game loop in `src/core`, the impact harvester reading solver impulses, and the noise model and wake meter moved out of the single file onto the modular physics, in one room layout.
- **Owner AI.** The staged wake state machine, a vision cone that uses our line-of-sight queries, and the caught rule. Then the behaviour tree (investigate the last noise, search, check hiding spots, return to bed), A* over hand-placed waypoints, and hearing.
- **The two remaining proofs (planned for Oct 28).** Coupling, and performance with every proof running, in Chrome and Firefox. The painted pass has to cost about half of what it does now, or be off by default.
- **Game around the slice.** HUD with the wake meter and chaos score, round end (caught, or a 3-minute timer), the shared debug overlay, and the cat's silhouette drawn through furniture.
- **Build and deploy.** Vite project, CI running the headless tests, and the live URL.
- **Open physics items.** Whether the cat's own weight should break plates (landing on one delivers 6.7 N·s; a plate breaks at 0.7). The cat sinks up to 1.6 cm into the floor for a frame when it lands. Shard colliders are simpler than the shards we draw.
- **Only if time allows:** sound, difficulty settings, replay.